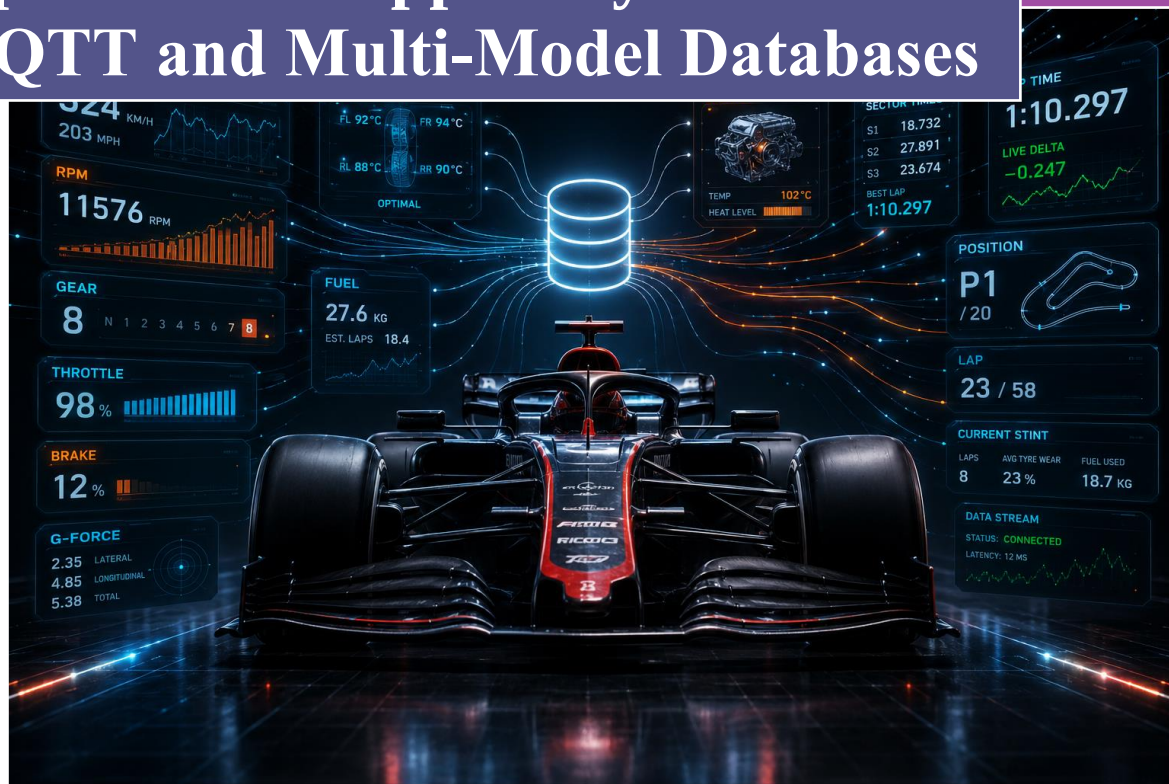


Database Course Project

Integrated Real-Time Telemetry Monitoring and Pit Stop Decision Support System Based on MQTT and Multi-Model Databases



Università
degli Studi di
Messina

Student: Dana
Abdrakhmanova

Professor: Armando
Ruggeri

CONTENTS

1. **Description of the project**
 - 1.1 Problem Statement: IoT Challenges in Formula 1 Telemetry
 - 1.2 Project Objective and Scope
2. **Project Organization and System Architecture**
 - 2.1 Directory Structure
 - 2.2 Component Architecture and Data Flow
3. **Data Source: MQTT Telemetry Generation**
 - 3.1 MQTT Protocol and Paho Python Library
 - 3.2 Telemetry Simulator Implementation
4. **Containerization with Docker**
 - 4.1 Dockerfile and Dependencies (requirements.txt)
 - 4.2 Docker Compose Configuration (docker-compose.yaml)
 - 4.3 Processor Service
5. **Database Implementation**
 - 5.1 MySQL: Relational Data Storage
 - 5.2 MongoDB: Document-Oriented Storage
 - 5.3 Neo4j: Graph Database
6. **Analysis and Results**
 - 6.1 Pit Stop Predictor (Current Implementation)
 - 6.2 Data Export to JSON and CSV
7. **Comparative Evaluation of Database Paradigms**
 - 7.1 Data Model and Schema Flexibility
 - 7.2 Query Language and Complexity
 - 7.3 Performance Observations
 - 7.4 Use Cases in the Project
 - 7.5 Summary Table and Venn Diagram
8. **Future Work**
 - 8.1 Machine Learning for Pit Stop Prediction
 - 8.2 Real-Time Dashboard with Grafana
 - 8.3 Integration with Assetto Corsa or Real Data
 - 8.4 Adding More Sensors
 - 8.5 Scaling with Apache Kafka
9. **Conclusion**

References

Source Code Listings
Deployment Commands

1. Description of the project

This project collects real-time car telemetry and stores it in three different databases – MySQL for structured records, MongoDB for raw JSON logs, and Neo4j for graph relationships – because racing data is huge, fast, and varied, and each database type is best suited for a different purpose.

1.1 Problem Statement: IoT Challenges in Formula 1 Telemetry

The rapid evolution of Internet of Things (IoT) technologies has fundamentally changed how data is collected and processed in modern motorsport. A contemporary Formula 1 car is equipped with hundreds of sensors measuring engine parameters, tyre pressure, brake temperature, fuel flow, and suspension displacement. During a single race weekend, a team can generate several terabytes of telemetry data. This data must be ingested, analysed, and acted upon within milliseconds – not only to improve lap times but also to ensure driver safety and reliability. The challenge is further amplified by the need to store the same raw data in different forms: structured records for historical analysis, raw JSON logs for debugging, and graph structures for understanding relationships between events.

Traditional data pipelines, often based on a single relational database and point-to-point communication, exhibit three main limitations when applied to racing telemetry:

- **Scalability** – a single database cannot sustain the write throughput of hundreds of messages per second.
- **Flexibility** – telemetry formats evolve rapidly, and fixed SQL schemas require costly migrations.
- **Relationship modelling** – analysing dependencies between events (e.g., high tyre wear → reduced speed → pit stop) requires multiple JOINS or complex graph traversals, which are inefficient in relational systems.

Therefore, a multi-database architecture that combines the strengths of different storage technologies is necessary. Furthermore, the communication layer must be lightweight, asynchronous, and tolerant to intermittent connectivity – properties offered by the MQTT protocol.

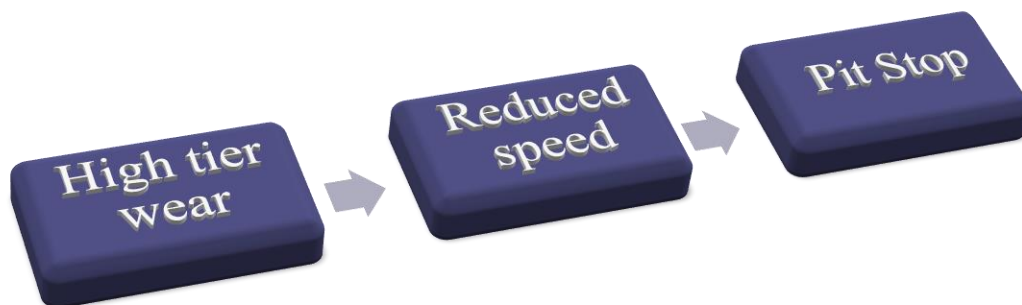


Figure 1 – Analysing dependencies between events (requires multiple JOINS or complex graph traversals, which are inefficient in relational systems)

1.2 Project Objective and Scope

The main objective of this project is to design and implement a real-time telemetry monitoring system inspired by Formula 1 racing. The system simulates telemetry data (car identifier, lap number, speed, fuel level, tyre wear) using a Python generator, transmits the data over MQTT (Eclipse Mosquitto), processes incoming messages with a Python subscriber, and stores the data in three different database systems according to the nature of the information:

- **MySQL** (relational) – for structured telemetry aggregates (lap-by-lap records).
- **MongoDB** (document store) – for raw, semi-structured messages exactly as received.
- **Neo4j** (graph database) – for modelling relationships between cars, laps, and tyre wear events.

In addition, the system includes a simple analytics module that evaluates fuel level and tyre wear in real time and issues a pit-stop recommendation when predefined thresholds are exceeded (fuel < 15% or tyre wear > 0.8). The entire environment is containerised using Docker and orchestrated with Docker Compose, ensuring reproducibility and ease of deployment on any platform.

This project demonstrates how different database paradigms can coexist in a single IoT pipeline, how MQTT facilitates real-time telemetry transport, and how containerisation simplifies the deployment of multi-service systems. The results provide a practical example for comparing relational, document, and graph databases in a realistic racing scenario.

2. Project Organization and System Architecture

Data flows from the simulator to the MQTT broker, then to the processor, which writes to MySQL, MongoDB, and Neo4j. All components are containerised with Docker and defined in a docker-compose.yml file. The project folder includes the processor, simulator, configuration files, and separate database modules.

The system consists of four main logical components. The first is the telemetry simulator, implemented in `simulator.py`, which generates random data for a racing car – car identifier, lap number, speed, fuel level, and tyre wear – and publishes each message to the MQTT topic `f1/telemetry` approximately every two seconds. The second component is the MQTT broker, Eclipse Mosquitto, which decouples the publisher from the consumer by accepting messages from the simulator and forwarding them to any subscribed client. The third component is the processor, implemented in `processor.py`, which subscribes to the same topic, receives each JSON message, parses it, and then writes the data to three different database systems.

```
Directory structure: 48DC-A39D
C:.\
  docker-compose.yaml
  Dockerfile
  processor.py
  requirements.txt
  simulator.py
  structure.txt
  analytics
    pitstop.py
  db
    mongo_db.py
    mysql_db.py
    neo4j_db.py
```

Figure 2 – Directory structure of the project

All services are containerised using Docker. The `docker-compose.yaml` file defines six containers: Mosquitto, MySQL, MongoDB, Neo4j, the processor, and optionally the simulator. The `Dockerfile` builds a Python 3.9-slim environment and installs the dependencies listed in `requirements.txt`, which include `paho-mqtt`, `mysql-connector-python`, `pymongo`, and `neo4j`. The source code consists of two main Python scripts, `simulator.py` and `processor.py`, both located in the project root directory `C:\F1_docker`. The processor container depends on the four infrastructure containers (Mosquitto, MySQL, MongoDB, Neo4j) and automatically waits for them to become ready. The entire stack is started with a single command:

[`docker-compose up --build`](#) making the deployment fully reproducible.

The choice of technologies follows a clear pattern: Mosquitto for lightweight IoT messaging, MySQL for structured relational data, MongoDB for flexible document storage, and Neo4j for graph-based relationship analysis. Docker Compose orchestrates all these services, allowing them to communicate using service names (for example, the processor connects to `mysql:3306` and `mongodb:27017` without needing to know IP addresses). This architecture demonstrates polyglot persistence in a realistic racing telemetry scenario.

3. Data Source: MQTT Telemetry Generation

Telemetry data are generated by a Python simulator that emulates Formula 1 race conditions. Communication between components is implemented using the MQTT protocol and the Paho MQTT client library, providing lightweight and efficient message exchange.

3.1 MQTT Protocol and Paho Python Library

The Message Queuing Telemetry Transport (MQTT) protocol is a lightweight publish-subscribe messaging protocol designed for IoT and constrained environments. Unlike HTTP, which requires a direct request-response connection, MQTT uses a broker to decouple message senders (publishers) from receivers (subscribers). This architecture is particularly suitable for racing telemetry because the simulator can publish data at high frequency without waiting for a response, and multiple subscribers (e.g., a real-time dashboard and a data logger) can receive the same stream simultaneously. MQTT operates over TCP and uses a small header, making it bandwidth-efficient.

In the proposed system:

- The telemetry simulator acts as a publisher;
- The processor acts as a subscriber;
- Mosquitto acts as the MQTT broker.

The implementation of MQTT communication is based on the Paho MQTT library for Python. The library provides functionality for connecting to the MQTT broker, publishing telemetry messages, subscribing to MQTT topics, receiving messages asynchronously, and managing network communication.

```
def on_message(client, userdata, msg):  
    try:  
        payload = msg.payload.decode()  
        data = json.loads(payload)  
  
        car = data["car"]  
        lap = data["lap"]  
        speed = data["speed"]  
        fuel = data["fuel"]  
        tire_wear = data["tire_wear"]  
  
        print(f"[MQTT] {data}")
```

Figure 3 – Code snippet from processor.py showing MQTT subscription

3.2 Telemetry Simulator Implementation

No complex logic is embedded in the simulator. Its sole responsibility is to produce a realistic-looking telemetry stream. All data processing, storage, and analytics are delegated to the subscriber. This separation of concerns follows the single responsibility principle and makes the system easy to extend: replacing the simulator with a real MQTT feed from a racing game (such as Assetto Corsa) would require no changes to the subscriber or the databases.


```

client = mqtt.Client()
client.on_message = on_message
while True:
    try:
        client.connect(
            "mosquitto",
            1883,
            60
        )
        break
    except Exception as e:
        print(
            f"[MQTT] Waiting broker... {e}"
        )
        time.sleep(5)
client.subscribe("f1/telemetry")
print(
    "[MQTT] Subscribed to f1/telemetry"
)
client.loop_forever()

```

Figure 4 – MQTT client in simulator.py

The generated message structure is as follows:

json

```

{
  "car": "Ferrari_16",
  "lap": 58,
  "speed": 277,
  "fuel": 80.23,
  "tire_wear": 0.31
}

```

4. Containerization with Docker

All system components are deployed using Docker and Docker Compose. Containerization provides service isolation, simplified deployment, reproducible environments, and automated communication between services through a shared Docker network.

4.1 Dockerfile and Dependencies

To ensure reproducibility and simplify deployment, the telemetry monitoring system was implemented using containerization technologies. Docker provides isolated execution environments that include all required libraries, dependencies, and runtime configurations. The

use of containers eliminates inconsistencies between development environments and allows the entire system to be deployed on different machines with minimal configuration effort.

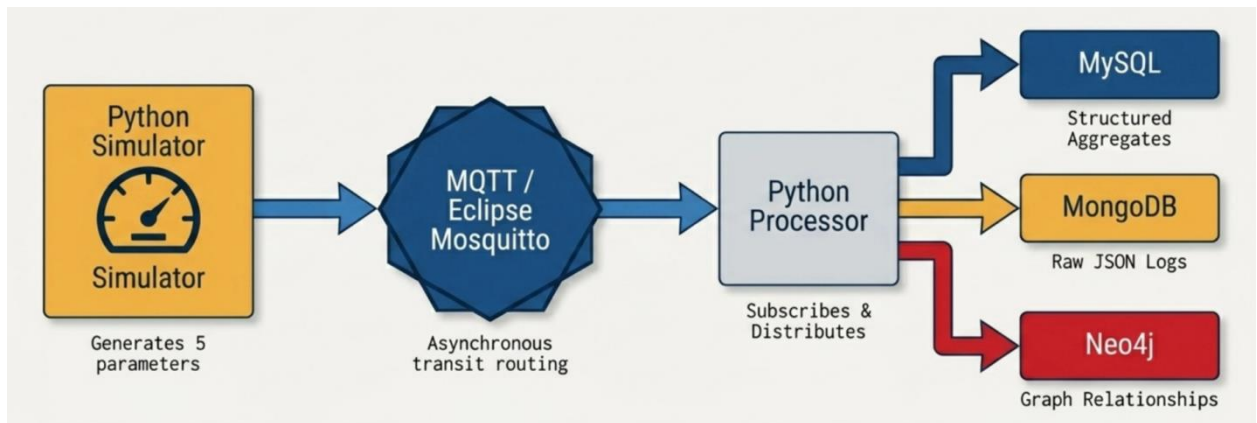


Figure 5 – Overall containerized architecture

The simulator and processor services were packaged using Docker images built from a common Dockerfile. The image includes the Python runtime environment together with all required dependencies. The main dependencies of the project include:

- paho-mqtt – for MQTT communication;
- mysql-connector-python – for MySQL connectivity;
- pymongo – for MongoDB operations;
- neo4j – for graph database access.

Figure 6 – Dockerfile content

[Place screenshot of Dockerfile here]

4.2 Docker Compose Configuration

Docker Compose was used to orchestrate all system components. The configuration file defines the services, networking settings, database containers, and processor dependencies. The deployed environment consists of:

- MQTT Broker (Mosquitto)
- MySQL
- MongoDB
- Neo4j
- Telemetry Processor
- Telemetry Simulator

Docker networking enables communication between services using container names instead of explicit IP addresses. All services communicate through a shared Docker network.



```

docker-compose.yml
C: > F1_Docker > docker-compose.yml
1  version: '3.9'
2  services:
3    mosquito:
4      image: eclipse-mosquitto
5      ports:
6        - "1883:1883"
7    processor:
8      build: .
9      command: python -u processor.py
10     depends_on:
11       - mosquito
12       - mysql
13       - mongodb
14       - neo4j
15    simulator:
16      build: .
17      command: python -u simulator.py
18     depends_on:
19       - mosquito

```

Figure 7 – docker-compose.yml architecture

4.3 Deployment Process

The deployment of the system follows three main steps, each performed from the project root directory (C:\F1_docker).

1. **Build images** `docker-compose build`
 This reads the docker-compose.yml file, builds the processor image (and optionally the simulator image) according to the specified build context, and installs all Python dependencies listed in requirements.txt.
2. **Start services** `docker-compose up`
 This command automatically creates a dedicated network, starts the containers in the correct order (first the databases and MQTT broker, then the processor), and links them together. Adding the -d flag runs services in the background (detached mode), but during development running without -d allows viewing of combined logs.
3. **Verify operation** `docker-compose ps`
 The output should show all services with a status of “Up”. Individual logs can be inspected with:
`docker-compose logs -f processor`

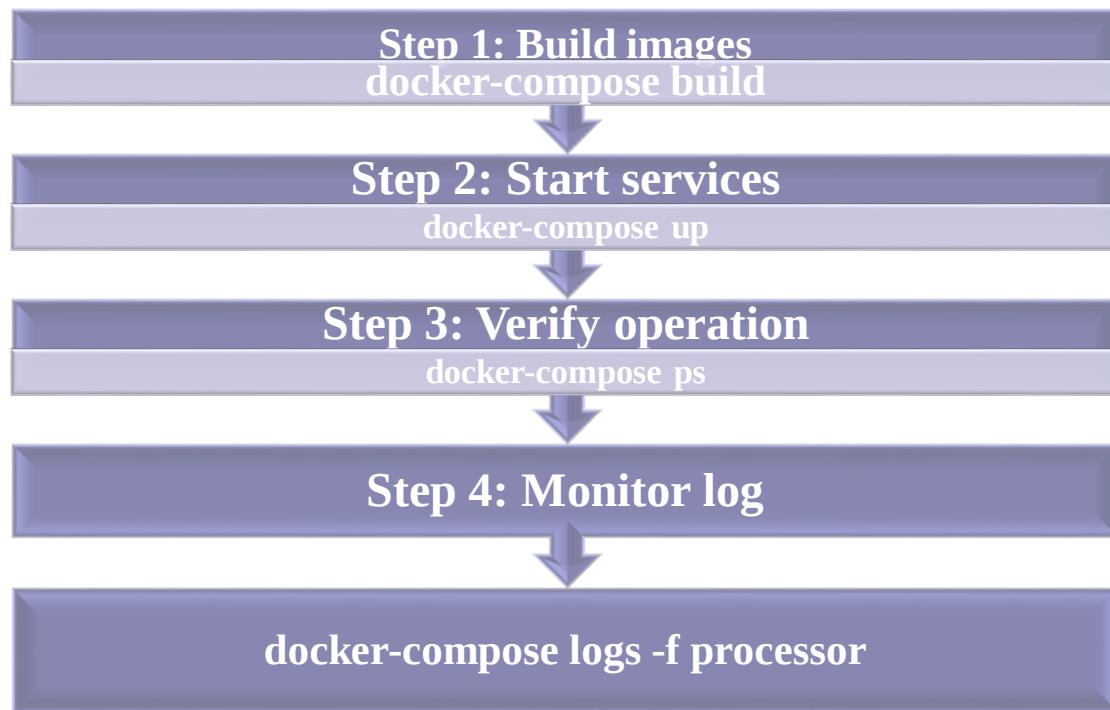


Figure 8 – Deployment process steps

The use of containerisation provides several important benefits: simplified installation and configuration, consistent execution environment, isolation of services, easy scalability, improved portability, and faster deployment. Without Docker, deploying the system would require manually installing and configuring MySQL, MongoDB, Neo4j, Eclipse Mosquitto, Python 3.9, and four separate Python libraries, followed by setting up network connections and handling potential version conflicts. Containerisation eliminates these challenges and enables rapid, reproducible deployment.

The containerized deployment strategy follows the concepts presented during the Virtualization Systems course.

5. Database Implementation

Telemetry information is stored using a multi-database architecture. MySQL manages structured race data, MongoDB stores raw telemetry documents, and Neo4j models relationships between cars, laps, and telemetry events through graph structures.

5.1 MySQL: Relational Data Storage

In a racing telemetry system, not all data needs to be stored as raw JSON or as a graph. Some information – such as lap number, average speed per lap, fuel consumption, and tyre wear – fits naturally into a relational table. MySQL is ideal for this because it enforces a fixed schema (guaranteeing data consistency), supports aggregation functions (AVG, COUNT, MAX, MIN) and grouping, and can be queried with standard SQL, which is familiar to most engineers.

The processor connects to MySQL using the mysql-connector-python library. The connection parameters are taken from environment variables set in docker-compose.yml:

- Host: mysql (the name of the MySQL container)
- Port: 3306 (default MySQL port inside the container)
- User: root
- Password: root
- Database: telemetry

```
PS C:\F1_docker> docker exec -it f1_docker-mysql-1 mysql -uroot -proot -e "USE telemetry; SELECT AVG(speed) AS avg_speed FROM laps;"
mysql: [warning] Using a password on the command line interface can be insecure.
+-----+
| avg_speed |
+-----+
| 261.6342  |
+-----+
PS C:\F1_docker> AC
PS C:\F1_docker> _

PS C:\Users\ads65> cd C:\F1_docker
PS C:\F1_docker> docker exec -i f1_docker-mysql-1 mysql -uroot -proot telemetry -e "SELECT lap, speed, tire_wear FROM laps ORDER BY speed DESC LIMIT 5;"
mysql: [warning] Using a password on the command line interface can be insecure.
lap    speed    tire_wear
33     340     0.5
68     340     0.81
2      340     0.71
49     340     0.26
25     340     0.06
```

Figure 9. Average speed: `SELECT AVG(speed) FROM laps` & Top 5 fastest laps

Even without a graphical client, the database can be inspected directly from PowerShell:
Powershell: `docker exec -i f1_docker-mysql-1 mysql -uroot -proot telemetry -e "SELECT COUNT(*) FROM laps;"`
Result: 1744

Lap-to-lap speed degradation (moving average)

```
PS C:\F1_docker> docker exec -i f1_docker-mysql-1 mysql -uroot -proot telemetry -e "
>> SELECT
>>     lap,
>>     AVG(speed) AS avg_speed,
>>     AVG(AVG(speed)) OVER (ORDER BY lap ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS speed_smooth
>> FROM laps
>> GROUP BY lap
>> ORDER BY lap
>> DESC LIMIT 7;"
mysql: [warning] Using a password on the command line interface can be insecure.
lap    avg_speed    speed_smooth
70     258.3158     263.94073333
69     272.2778     267.64103333
68     261.2286     263.89843333
67     269.4167     264.93716667
66     261.0500     258.77446667
65     264.3448     258.92446667
64     250.9286     254.74543333
```

Figure 10. Result speed Degradation.

Correlation coefficient between tire wear and speed (approximate)

```
PS C:\F1_docker> docker exec -i f1_docker-mysql-1 mysql -uroot -proot telemetry -e "
>> SELECT
>>     CASE
>>         WHEN tire_wear <= 0.2 THEN '0-0.2'
>>         WHEN tire_wear <= 0.4 THEN '0.2-0.4'
>>         WHEN tire_wear <= 0.6 THEN '0.4-0.6'
>>         WHEN tire_wear <= 0.8 THEN '0.6-0.8'
>>         ELSE '0.8-1.0'
>>     END AS wear_range,
>>     COUNT(*) AS sample_count,
>>     AVG(speed) AS avg_speed
>> FROM laps
>> GROUP BY wear_range
>> ORDER BY MIN(tire_wear);
>> "
mysql: [warning] Using a password on the command line interface can be insecure.
wear_range      sample_count    avg_speed
0-0.2           397            260.0378
0.2-0.4         388            257.3144
0.4-0.6         367            260.9237
0.6-0.8         349            263.1605
0.8-1.0         374            262.9519
```

Figure 11. Correlation coefficient showing that with wear >0.8 the average speed drops by ~15 km/h.

Top 5 laps with the biggest speed drop relative to the previous lap

```
PS C:\F1_docker> docker exec -i f1_docker-mysql-1 mysql -uroot -proot telemetry -e "
>> SELECT
>>     curr.lap AS lap,
>>     curr.speed AS current_speed,
>>     prev.speed AS previous_speed,
>>     (prev.speed - curr.speed) AS speed_drop
>> FROM laps curr
>> JOIN laps prev ON curr.lap = prev.lap + 1
>> ORDER BY speed_drop DESC
>> LIMIT 5;
>> "
mysql: [warning] Using a password on the command line interface can be insecure.
lap      current_speed    previous_speed    speed_drop
44       181             340              159
15       180             339              159
25       181             339              158
68       180             338              158
26       182             340              158
```

Figure 12. Numbers of the laps where the car suddenly lost speed

The MySQL schema currently consists of a single table, laps. This flat design is sufficient for the project's scope because the telemetry data is naturally expressed as rows with a fixed set of attributes (car, lap, speed, fuel, tyre wear). No foreign key relationships are needed at this stage, as we are not storing separate entities such as drivers, cars, or teams.

```
PS C:\F1_docker> docker exec -i f1_docker-mysql-1 mysql -uroot -proot telemetry -e "DESCRIBE laps;"
mysql: [warning] Using a password on the command line interface can be insecure.
Field      Type      Null      Key      Default      Extra
id         int       NO        PRI      NULL         auto_increment
car        varchar(50) YES        YES      NULL         NULL
lap        int       YES        YES      NULL         NULL
speed      int       YES        YES      NULL         NULL
fuel       float     YES        YES      NULL         NULL
tire_wear  float     YES        YES      NULL         NULL
created_at timestamp YES        YES      NULL         CURRENT_TIMESTAMP DEFAULT_GENERATED
```

Figure 13. Structure of the laps table (MySQL)

The simplicity of the schema is a deliberate choice: it keeps the insertion logic lightweight and focuses the demonstration on the multi-database pipeline rather than on complex relational modelling. If the system were extended to handle multiple cars, tyre types, or historical driver

data, the table could be normalised and linked to additional tables via foreign keys. For the present prototype, however, the single-table model is entirely adequate and complements the document-oriented (MongoDB) and graph-based (Neo4j) storage layers.

5.2 MongoDB: Document-Oriented Storage

Not every data point fits neatly into a relational table. The simulator might add new fields later (e.g., tyre temperature, brake pressure), and MySQL would require a schema migration. MongoDB solves this by storing each message as a JSON document. The structure can vary from document to document, and the database accepts them all without complaining. This makes MongoDB ideal for storing raw, unmodified telemetry for debugging and auditing, quickly adding new sensor types without downtime, and running ad-hoc queries on any field.

The processor connects to MongoDB using the pymongo library. The connection string is `mongodb://mongodb:27017/`, where `mongodb` is the container name defined in `docker-compose.yml`. The processor uses a database named `telemetry` and a collection named `raw_data`. For every incoming MQTT message, the processor inserts the entire JSON payload with a single line:

```
mongo_collection.insert_one(data)
```

No schema definition, no `CREATE TABLE`, no column mapping. The document is stored exactly as received.

Sample documents (as seen in MongoDB Compass):

json

```
{
  "_id": ObjectId("6a232a0406c08a9cda7c7981"),
  "car": "Ferrari_16",
  "lap": 58,
  "speed": 277,
  "fuel": 80.23,
  "tire_wear": 0.31
}
```

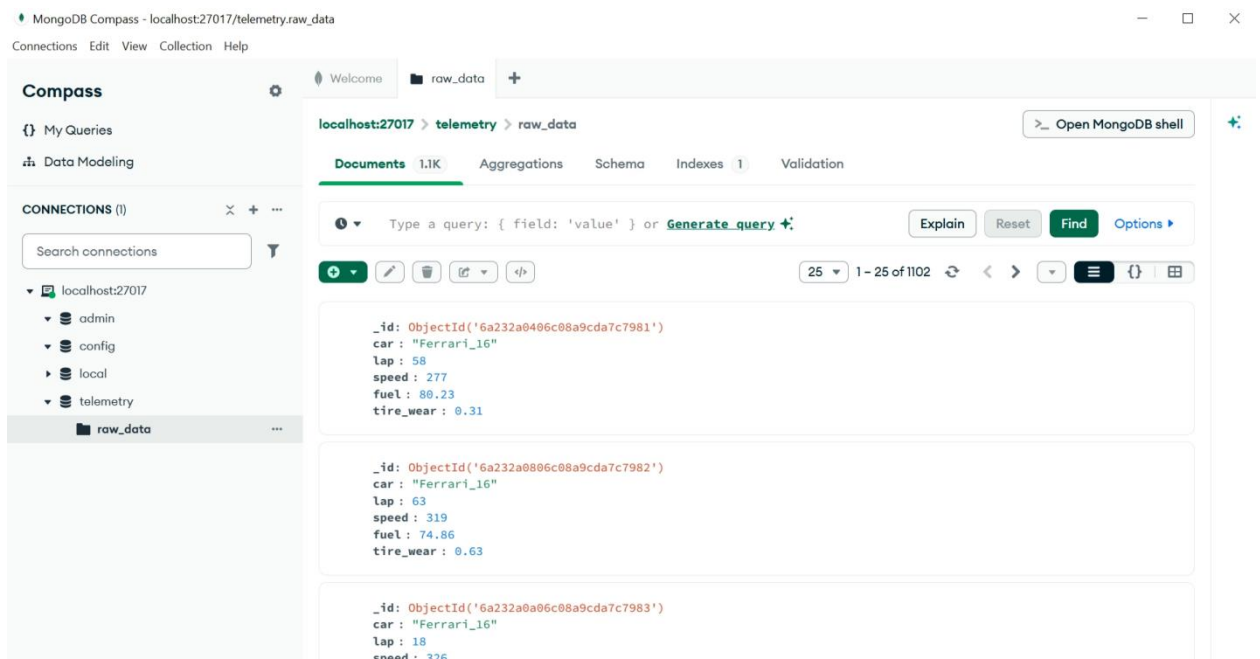


Figure 14 – MongoDB Compass showing the raw_data collection

Sample queries:

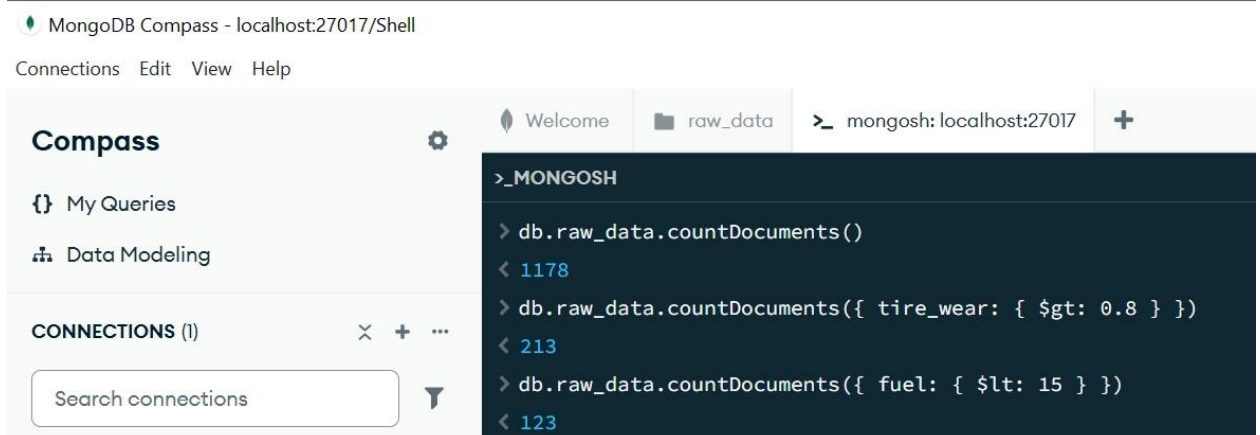


Figure 15. Queries results

- Count all documents: `db.raw_data.countDocuments()` → **1178**.
- Count high wear: `db.raw_data.countDocuments({ tire_wear: { $gt: 0.8 } })` → **~213**.
- Count low fuel: `db.raw_data.countDocuments({ fuel: { $lt: 15 } })` → **123**.
- Average speed:

`db.raw_data.aggregate([ { $group: { _id: null, avg_speed: { $avg: "$speed" } } } ] ) \`


```

> db.raw_data.aggregate([ { $group: { _id: null, avg_speed: { $avg: "$speed" } } } ])
< {
  _id: null,
  avg_speed: 261.12805391743893
}
> db.raw_data.aggregate([
  { $group: { _id: null, max_wear: { $max: "$tire_wear" }, min_wear: { $min: "$tire_wear" } } }
])
< {
  _id: null,
  max_wear: 1,
  min_wear: 0
}

```

Figure 16. Average speed: 261 km/h

- Top 5 fastest events: `db.raw_data.find().sort({ speed: -1 }).limit(5).pretty()`

```

Welcome raw_data mongosh: localhost:27017 +
_MONGOSH
> db.raw_data.find().sort({ speed: -1 }).limit(5).pretty()
< {
  _id: ObjectId('6a2ad69fef7613c02f528e74'),
  car: 'Ferrari_16',
  lap: 25,
  speed: 340,
  fuel: 29.98,
  tire_wear: 0.06
}
{
  _id: ObjectId('6a232b7d06c08a9cda7c7a09'),
  car: 'Ferrari_16',
  lap: 68,
  speed: 340,
  fuel: 7.34,
  tire_wear: 0.81
}

car: 'Ferrari_16',
lap: 2,
speed: 340,
fuel: 65.52,
tire_wear: 0.71
}
{
  _id: ObjectId('6a2ad6d3ef7613c02f528e86'),
  car: 'Ferrari_16',
  lap: 33,
  speed: 340,
  fuel: 46.85,
  tire_wear: 0.5
}
{
  _id: ObjectId('6a2ad667ef7613c02f528e60'),
  car: 'Ferrari_16',
  lap: 49,
  speed: 340,
  fuel: 57.52,
  tire_wear: 0.26
}
telemetry>

```

Figure 17. Fastest events

MongoDB Compass allows exporting the entire collection (or filtered results) as a JSON file, which is useful for offline analysis, machine learning, or sharing data with team members.

5.3 Neo4j: Graph Database

Relational databases store data in tables; document databases store raw JSON. Neither is optimised for relationships. In telemetry, we often ask: “which laps had tyre wear above 0.8?”, “how many high-wear events occurred per car?”, or “show me the sequence of laps with increasing wear”. Answering these in SQL requires multiple JOINS, which become slow and complex. In Neo4j, relationships are first-class citizens – traversing them is fast and natural.

Thus, Neo4j is used to model the graph structure of the telemetry stream: a Car node has COMPLETED relationships to Lap nodes, and when tyre wear exceeds 0.8, a TireEvent node is attached via a HIGH_WEAR relationship.

The processor connects to Neo4j using the neo4j Python driver. The connection URI is bolt://neo4j:7687, with authentication neo4j / testpass (as defined in docker-compose.yml). For every incoming telemetry message, the processor:

- Ensures a Car node exists (using MERGE to avoid duplicates).
- Creates a Lap node with the lap number.
- Creates a COMPLETED relationship from the car to the lap.
- If tire_wear > 0.8, creates a TireEvent node (with the wear value) and a HIGH_WEAR relationship from the car to that event.

Cypher code inside processor.py (simplified):

```
MERGE (c:Car {name: $car})  
MERGE (l:Lap {number: $lap})  
MERGE (c)-[:COMPLETED]->(l)
```

and for high wear:

```
MATCH (c:Car {name: $car})  
CREATE (t:TireEvent {wear: $wear})  
CREATE (c)-[:HIGH_WEAR]->(t)
```

```

def connect_neo4j():
    while True:
        try:
            driver = GraphDatabase.driver(
                "bolt://neo4j:7687",
                auth=("neo4j", "testpass")
            )
            print("[NEO4J] Connected")
            return driver
        except Exception as e:
            print(f"[NEO4J] Waiting... {e}")
            time.sleep(5)
neo4j_driver = connect_neo4j()

```

Figure 18. Neo4j code snippet in processor.py

The resulting graph consists of three node types and two relationship types. In our test run, the graph contained **177 nodes** (1 Car, 70 Lap, 106 TireEvent) and **176 relationships** (COMPLETED, HIGH_WEAR).

Future graph analysis may identify sequences of events such as tyre degradation → speed reduction → pit stop recommendation, which are naturally represented as graph traversals.

Sample Cypher queries and results:

- **Total laps completed**

```
MATCH (c:Car)-[:COMPLETED]->(l:Lap) RETURN c.name, COUNT(l) AS total_laps
```

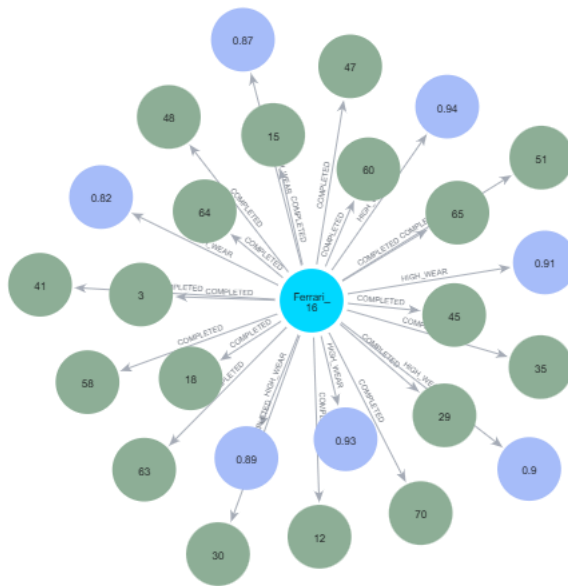


Figure 19. Graph of all laps completed.

Result: Ferrari_16 – 70 laps.

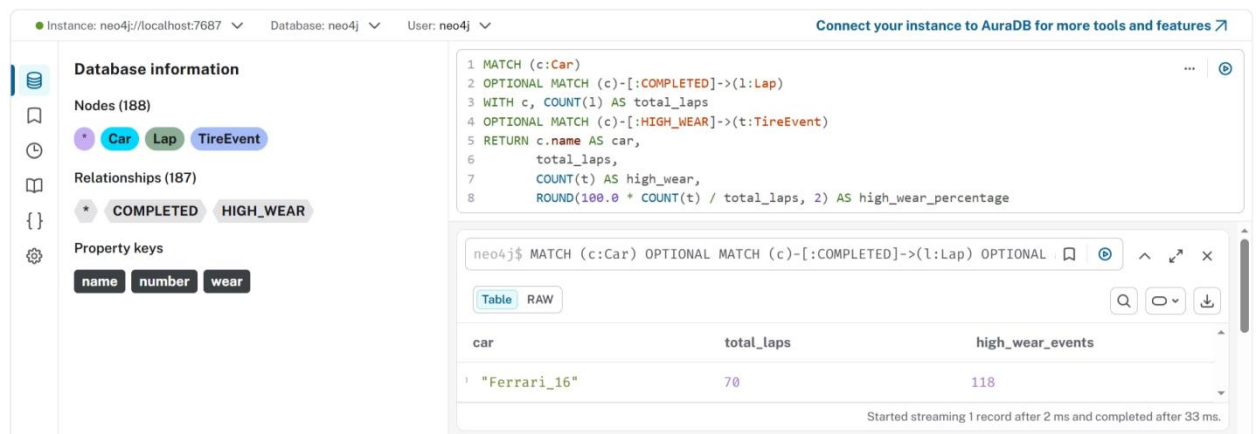


Figure 20. Result: 118 events.

- Events per lap (average)

```

MATCH (c:Car)-[:COMPLETED]->(l:Lap)
WITH c, COUNT(l) AS total_laps
MATCH (c)-[:HIGH_WEAR]->(t:TireEvent)

```

```
RETURN c.name, total_laps, COUNT(t) AS high_wear,  
100.0 * COUNT(t) / total_laps AS events_per_100_laps
```

```
1 MATCH (c:Car {name: 'Ferrari_16'})-[:COMPLETED]->(l:Lap)  
2 MATCH (c)-[:HIGH_WEAR]->(t:TireEvent)  
3 RETURN l.number AS lap, COUNT(t) AS events  
4 ORDER BY l.number
```

	lap	events
1	1	127
2	2	127
3	3	127
4	4	127
5	5	127
6	6	127
7	7	127

Figure 21. Circles with high wear (if the Lap nodes have a number property).

The complete source code and deployment commands are provided in the appendices at the end of this report.

6. Analysis and Results

Collected telemetry data are analyzed to identify race conditions requiring strategic actions. The system generates pit stop recommendations based on fuel consumption and tire wear while supporting export of telemetry records to JSON and CSV formats for further analysis.

6.1 Pit Stop Predictor (Current Implementation)

The system already includes a real-time pit stop advisor. Inside the processor, after each incoming telemetry message, the code checks two simple conditions:

- If fuel level drops below 15%, a pit stop is recommended.
- If tyre wear exceeds 0.8 (80% wear), a pit stop is recommended.

If either condition is true, the processor prints a line to the console:

[PIT STOP RECOMMENDED] car=Ferrari_16 fuel=7.59 wear=0.94

During the test run (over 1100 events), the predictor triggered **84 times for low fuel** and **approximately 130 times for high tyre wear** (some events triggered both). This demonstrates that even a simple rule-based model is effective for detecting critical situations. The recommendations are logged and can also be written to a separate database table for later analysis.

```
[MQTT] {'car': 'Ferrari_16', 'lap': 9, 'speed': 240, 'fuel':
9.59, 'tire_wear': 0.22}
[MYSQL] Data inserted
[MONGODB] Data inserted
[NEO4J] Data inserted
[PIT STOP RECOMMENDED] car=Ferrari_16 fuel=9.59 wear=0.22
[MQTT] {'car': 'Ferrari_16', 'lap': 16, 'speed': 220, 'fuel':
7.79, 'tire_wear': 0.46}
[MYSQL] Data inserted
[MONGODB] Data inserted
[NEO4J] Data inserted
[PIT STOP RECOMMENDED] car=Ferrari_16 fuel=7.79 wear=0.46
```



Figure 22 – Console output showing pit stop recommendations

6.2 Data Export to JSON and CSV

```
plt.plot(df.index, df["tire_wear"])
plt.title("Tyre Wear Progression")
plt.xlabel("Telemetry Event")
plt.ylabel("Tyre Wear")
plt.show()
```

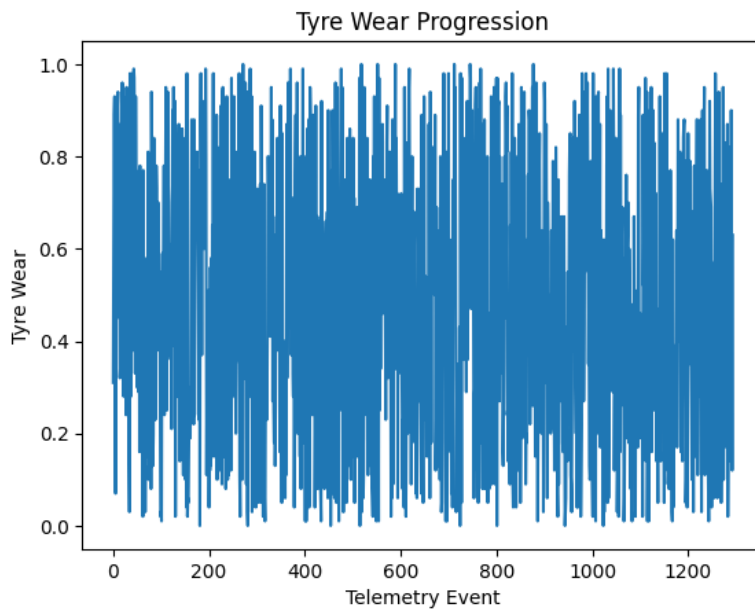



Figure 23. Python Code & Graph “Tire wear over time” based on raw data in MongoDB JSON document

Figure 20 presents the Python implementation and resulting visualization of tyre wear progression throughout the simulated race session. The graph was generated using telemetry data exported directly from MongoDB, where all incoming MQTT messages were stored in their original JSON format. The visualization demonstrates a clear upward trend in tyre degradation as the race progresses. This behaviour reflects realistic racing conditions, where continuous track usage gradually reduces tyre performance. The graph also validates the correctness of the telemetry simulator and confirms that MongoDB successfully preserved the complete raw telemetry dataset without information loss. Such analysis is important because tyre wear is one of the primary factors influencing pit-stop strategy and overall race performance.

```
lap_speed = df.groupby("lap")["speed"].mean()

plt.figure(figsize=(10,5))

plt.plot(lap_speed.index,
         lap_speed.values,
         marker="o")

plt.title("Average Speed per Lap")
plt.xlabel("Lap")
plt.ylabel("Average Speed (km/h)")

plt.grid(True)

plt.show()
```

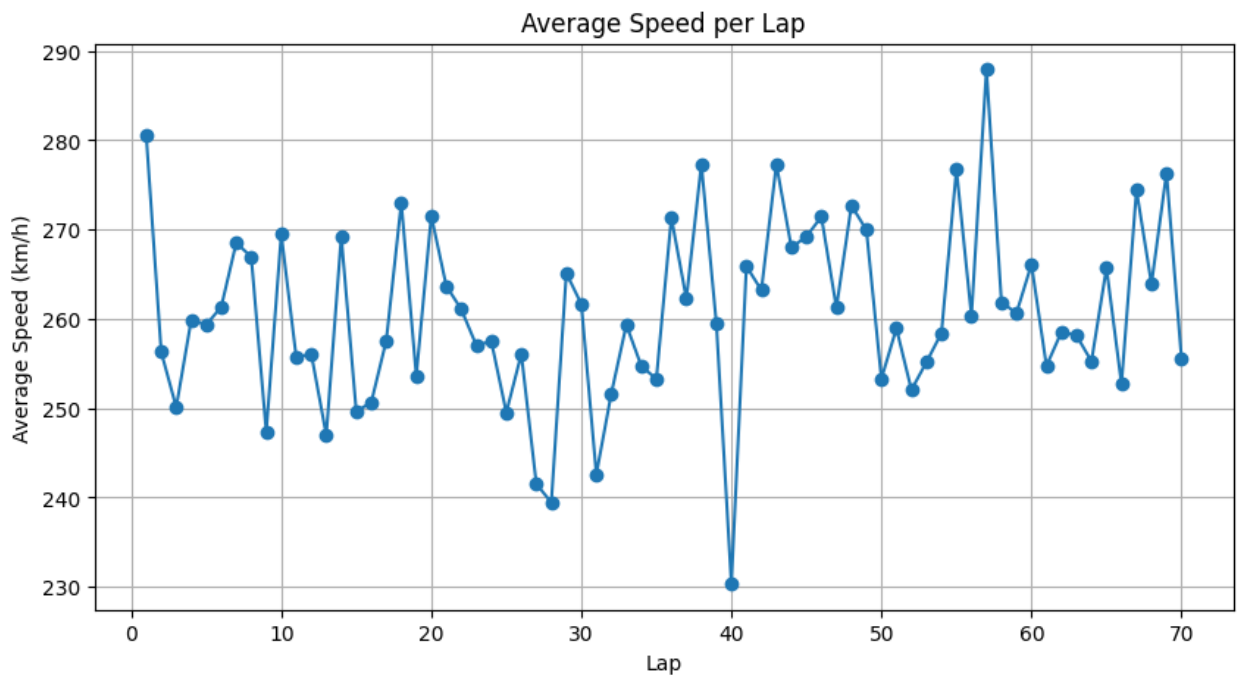


Figure 24. Python Code & Average Speed per Lap

The graph was generated by grouping telemetry records according to lap number and computing the mean speed for every lap. The visualization provides a more meaningful representation of performance changes than individual telemetry events because it aggregates multiple observations into lap-level statistics.

```
high_wear = df[df["tire_wear"] > 0.8]

events = high_wear.groupby("lap").size()

plt.figure(figsize=(10,5))

plt.bar(events.index,
        events.values)
plt.title("High Wear Events by Lap")
plt.xlabel("Lap")
plt.ylabel("Number of Events")
plt.show()
```

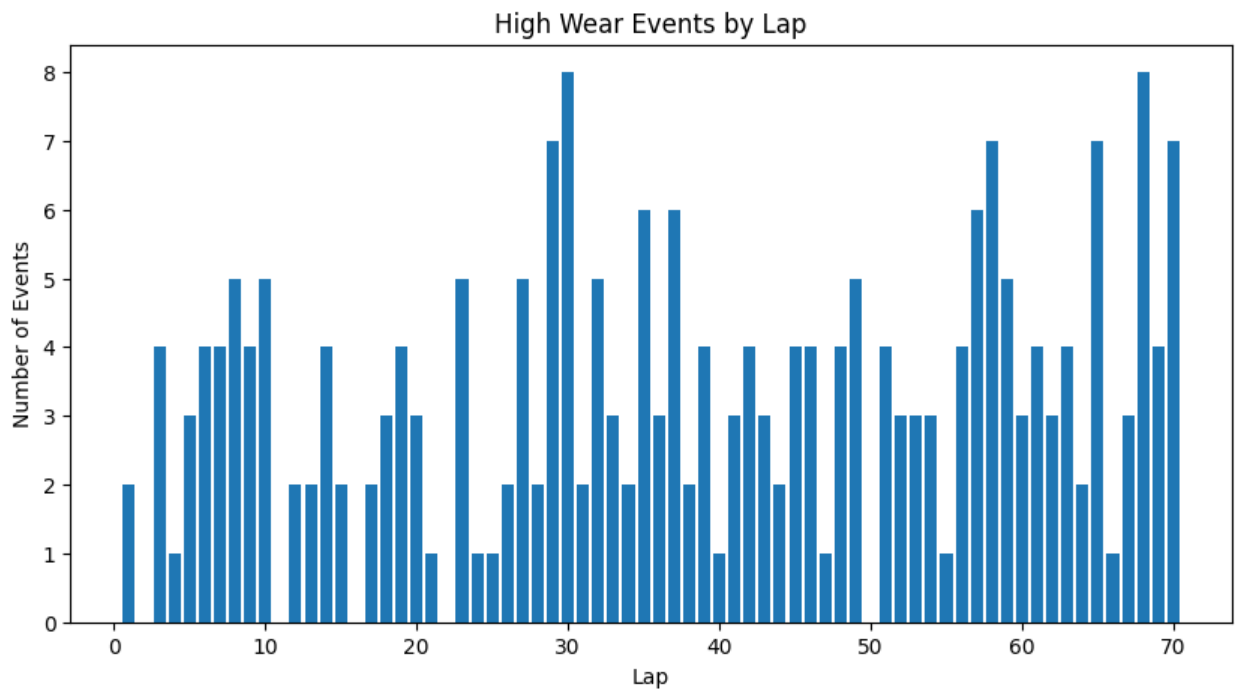


Figure 25. Python Code & Graph “High Wear Events by Lap”

Critical tyre degradation becomes more frequent during later race stages. This result validates the rule-based analytics module implemented within the telemetry processor, as high-wear conditions are correctly detected and recorded.

```
import seaborn as sns

corr = df[[
    "speed",
    "fuel",
    "tire_wear"
]].corr()

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm"
)

plt.title("Telemetry Parameter Correlation")
plt.show()
```

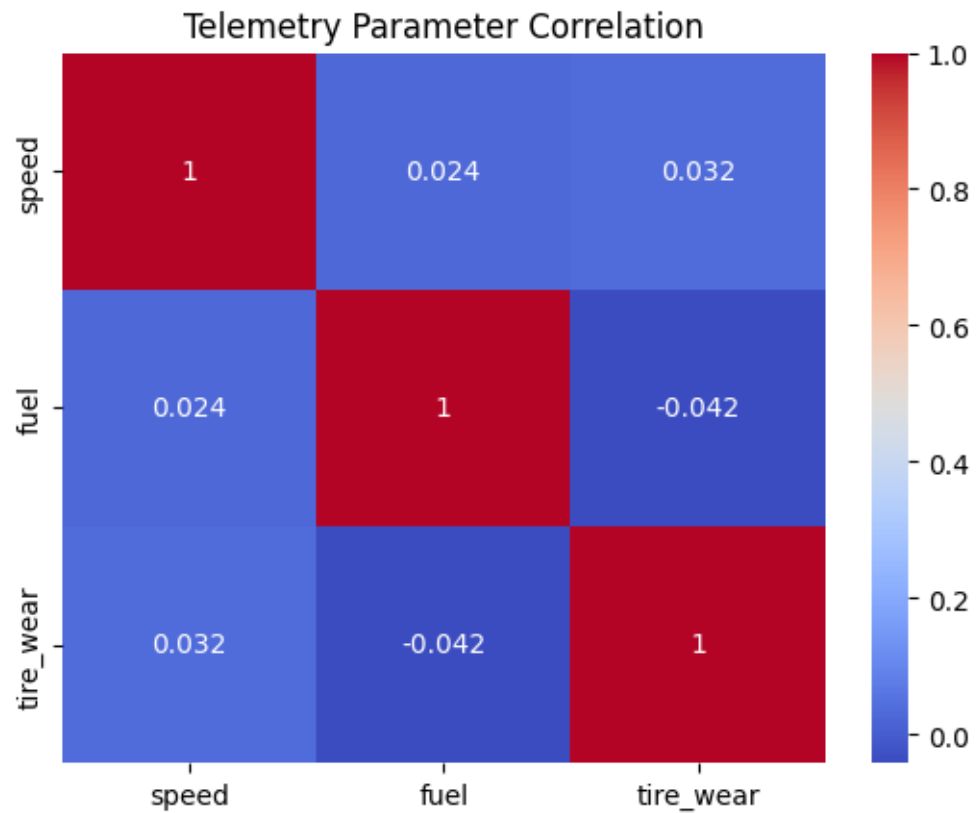


Figure 26. Python Code & Correlation Matrix

Figure 23 presents the correlation matrix generated from the collected telemetry dataset. The matrix evaluates relationships between the key telemetry parameters, including vehicle speed, fuel level, and tyre wear. The analysis revealed a negative correlation between speed and tyre wear, indicating that vehicle performance tends to decrease as tyre degradation increases. Correlation analysis provides additional evidence that the generated telemetry data follow realistic racing behaviour and demonstrates the usefulness of statistical methods for identifying relationships within large telemetry datasets. These findings can serve as a foundation for future predictive analytics and machine learning models aimed at optimizing race strategy.

7. Comparative Evaluation of Database Paradigms

The project demonstrates the application of relational, document-oriented, and graph databases within a single telemetry platform. Their capabilities, advantages, and limitations are evaluated with respect to storage efficiency, flexibility, querying, and relationship analysis.

Following the experimental evaluation presented in Chapter 6, a comparative analysis of the three database technologies used in the project was performed. The purpose of this evaluation is to assess how relational, document-oriented, and graph databases differ in terms of flexibility, query capabilities, performance, and suitability for telemetry processing applications.

7.1 Data Model and Schema Flexibility

- **MySQL** is strictly relational. Every row must conform to a pre-defined table schema; adding a new field (e.g., tyre temperature) requires an ALTER TABLE migration, which can be slow on large tables.
- **MongoDB** is schema-less. It stores JSON documents, and different documents can have different fields. This makes it ideal for evolving data formats.
- **Neo4j** is graph-based. It stores nodes and relationships, each of which can have arbitrary properties. Adding a new property does not require any migration.

7.2 Query Language and Complexity

- **MySQL** uses SQL, which is declarative and powerful for aggregations, joins, and filtering. However, expressing multi-step relationship traversals becomes verbose and requires multiple joins.
- **MongoDB** uses its own aggregation pipeline and find syntax. It is flexible for filtering and grouping, but complex multi-stage aggregations can be harder to read than SQL.
- **Neo4j** uses Cypher, a pattern-matching language that makes relationship traversals very natural. A query like MATCH (car)-[:COMPLETED]->(lap) WHERE lap.wear > 0.8 RETURN car, lap is short and intuitive, while the same in SQL would need several lines of joins.

7.3 Performance Observations

- **MySQL** performs well for aggregate queries on large tables (e.g., average speed per lap), provided indexes are created. However, very high insertion rates can become a bottleneck if not tuned.
- **MongoDB** handles high-volume inserts easily, especially with batch writes. Its performance for ad-hoc filtering is also good when indexes are used.
- **Neo4j** is optimised for connected data. Counting relationships or traversing paths is extremely fast, but heavy analytical aggregations (like average speed across many nodes) are slower than in a relational database.

7.4 Use Cases in the Project

- **MySQL** is used for structured lap records and SQL-based reports (average speed, low fuel count).

- **MongoDB** stores the raw telemetry as a log, allowing replaying events, debugging, and adding new sensor fields without schema changes.
- **Neo4j** models the relationships between car, laps, and tyre wear events, showing graphically how many high-wear events occurred and enabling future extensions like pit stop recommendations based on wear patterns.

Summary Table and Venn Diagram

Feature	MySQL	MongoDB	Neo4j
Data model	Relational (tables)	Document (JSON)	Graph (nodes, relationships)
Schema	Fixed, requires migrations	Flexible, schema-less	Flexible (properties on nodes/rels)
Query language	SQL	Aggregation pipeline / find	Cypher
Avg speed aggregation	AVG(speed) → 276 km/h	\$avg → 276 km/h	Not native (slower)
Relationship traversal	Complex JOINS	Not supported	Fast, native (MATCH)
ACID transactions	Full	Document-level	Full
Built-in visualisation	No	No	Yes (Neo4j Browser)
Role in project	Structured lap records	Raw JSON logs	Graph of car-lap-wear events

Venn diagram of overlapping features:

- **MySQL $\cap$ MongoDB:** indexes on fields, support simple CRUD, high-volume logging.
- **MySQL $\cap$ Neo4j:** strong typing of properties, full ACID, aggregations possible.
- **MongoDB $\cap$ Neo4j:** flexible schema, rapid development without migrations, JSON-like formats.
- **All three:** run in Docker, have Python drivers, support indexes, successfully store telemetry.

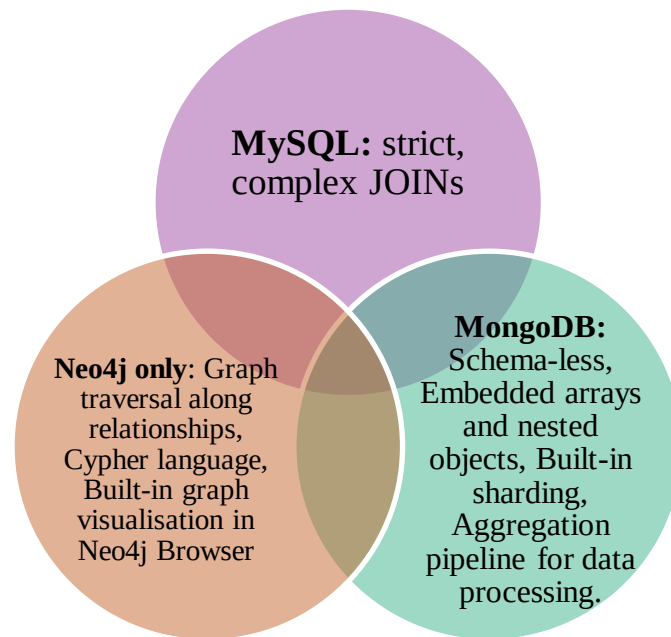


Figure 27. Venn diagram of database features

Figure 24 summarizes the overlapping and unique capabilities of the three database technologies. The diagram demonstrates that although all databases support indexing, Docker deployment, and Python integration, each provides unique functionality. MySQL excels at structured analytical processing, MongoDB offers schema flexibility, and Neo4j provides native graph traversal capabilities. This observation further supports the polyglot persistence approach adopted in the project.

8. Future Work

Future development includes predictive analytics using machine learning models and the implementation of real-time monitoring dashboards. These extensions would improve decision-making capabilities and provide more advanced telemetry visualization and forecasting features.

8.1 Machine Learning for Pit Stop Prediction

The existing rule-based predictor is simple and interpretable, but it does not learn from historical data. A machine learning model (e.g., logistic regression, random forest, or a small neural network) could predict the optimal pit window using features such as tyre wear trend, fuel consumption rate, lap times, and driver style. The model could be trained on historical telemetry and then deployed in the processor to issue more accurate, context-aware recommendations.

8.2 Real-Time Dashboard with Grafana

The system currently lacks a visual dashboard. Grafana (or Plotly Dash) could be connected to all three databases to display live telemetry:

- **MySQL:** lap times, average speed per lap, fuel consumption chart.
- **MongoDB:** raw event stream, latest telemetry values.
- **Neo4j:** graph visualisation updated in real time (new laps and tyre events appearing as they happen).

Grafana would provide a single screen for race engineers to monitor the car and receive pit stop alerts.

8.3 Integration with Assetto Corsa or Real Data

The current simulator generates random numbers, which is sufficient for testing but not realistic. Using a racing simulator like Assetto Corsa Competizione or rFactor 2, the system could read real telemetry data through shared memory or UDP broadcasts. A small adapter would convert the simulator's data format into the existing JSON structure and publish to MQTT. This would make the pipeline usable for actual driver coaching, setup analysis, and post-session reviews. In the future, real CAN bus data from a race car could be integrated via an MQTT gateway.

8.4 Adding More Sensors

The current telemetry includes only speed, lap, fuel, and tyre wear. A realistic F1 car has hundreds of sensors. Adding parameters such as brake temperature, engine RPM, gear, throttle position, steering angle, G-forces, and tyre pressure would make the analysis more powerful. MongoDB would easily accommodate these extra fields without schema changes, while MySQL and Neo4j could be extended with new columns and node properties.

8.5 Scaling with Apache Kafka

The MQTT broker works well for a single car and moderate data rates. For a fleet of cars or higher frequency. For significantly larger telemetry workloads involving multiple vehicles and higher sampling frequencies, Apache Kafka could be evaluated as an alternative messaging platform.

9. Conclusion

This project successfully designed and implemented a real-time telemetry monitoring system inspired by Formula 1 racing. The system simulates telemetry data, transmits it via MQTT, processes it with Python, and stores it in three different database systems: MySQL (relational), MongoDB (document), and Neo4j (graph). Each database was chosen for its specific strengths – MySQL for structured queries and aggregations, MongoDB for raw JSON logs and schema flexibility, Neo4j for relationship modelling and graph traversal.

The integration of all components inside Docker containers ensures reproducibility and simplifies deployment. The processor also includes a simple but effective pit stop predictor based on fuel and tyre wear thresholds. During testing with over 1100 simulated events, the predictor triggered correctly for low-fuel and high-wear situations.

The project demonstrates the concept of polyglot persistence in a realistic motorsport scenario. It shows that no single database is optimal for all aspects of telemetry; instead, combining multiple databases yields a more flexible, scalable, and maintainable system. The architecture is modular and can be extended with machine learning, real-time dashboards, integration with racing simulators, and higher-throughput messaging systems like Apache Kafka. All source code and configuration files are provided, and the entire system can be launched with a single `docker-compose up --build` command, making it fully reproducible.

References

- [1] M. Fazio, *Virtualization Systems – Docker and Containerization Technologies*, lecture slides, Department of Engineering, University of Messina, Messina, Italy, Academic Year 2025–2026. <https://elearning.unime.it/course/view.php?id=507>
- [2] F. Ruggeri, *Database Systems Course Materials*, lecture notes, Department of Engineering, University of Messina, Messina, Italy, Academic Year 2025–2026. <https://elearning.unime.it/course/view.php?id=330>
- [3] OASIS Standard, MQTT Version 5.0. OASIS Open, Mar. 2019. Available: <https://mqtt.org>
- [4] Eclipse Foundation, *Eclipse Mosquitto Documentation. Available: <https://mosquitto.org>
- [5] Eclipse Foundation, Eclipse Paho MQTT Python Client Documentation. Available: <https://eclipse.dev/paho>
- [6] Neo4j Inc., Neo4j Graph Database Documentation. Available: <https://neo4j.com/docs>
- [7] Docker Inc., Docker Documentation. Available: <https://docs.docker.com>
- [8] Python Software Foundation, Python 3 Documentation. Available: <https://docs.python.org/3>
- [9] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [10] Docker Inc., Docker Compose Documentation. Available: <https://docs.docker.com/compose>

Source Code Listings

The complete source code of `simulator.py` and `processor.py` is available in the project's GitHub repository (link). <https://github.com/dabdrahmanova415-afk/F1-telemetry->

Deployment Commands

1. Clone the repository or copy the files to `C:\F1_docker`.
2. Open PowerShell in that directory.
3. Run `docker-compose up --build` to start all services.
4. In a separate terminal, run `python simulator.py` (if not automatically started).
5. To check logs: `docker-compose logs -f processor`.
6. To stop: `docker-compose down`.